

LECTURE 11

머신러닝

경북대학교

전자공학부

김현수

목차

11.1 머신러닝의 개요

11.2 머신러닝 분류

11.3 머신러닝 회귀

11.4 [플러스 예제] MNIST 분류하기

학습목표

- 머신러닝의 개념을 설명할 수 있습니다.
- 머신러닝 절차를 설명할 수 있습니다.
- 머신러닝 분류 프로그램을 작성할 수 있습니다.
- 머신러닝 회귀 프로그램을 작성할 수 있습니다.

머신러닝의 개요

1. 머신러닝의 개요

■ 머신러닝의 개념

▪ 인공지능과 머신러닝, 딥러닝

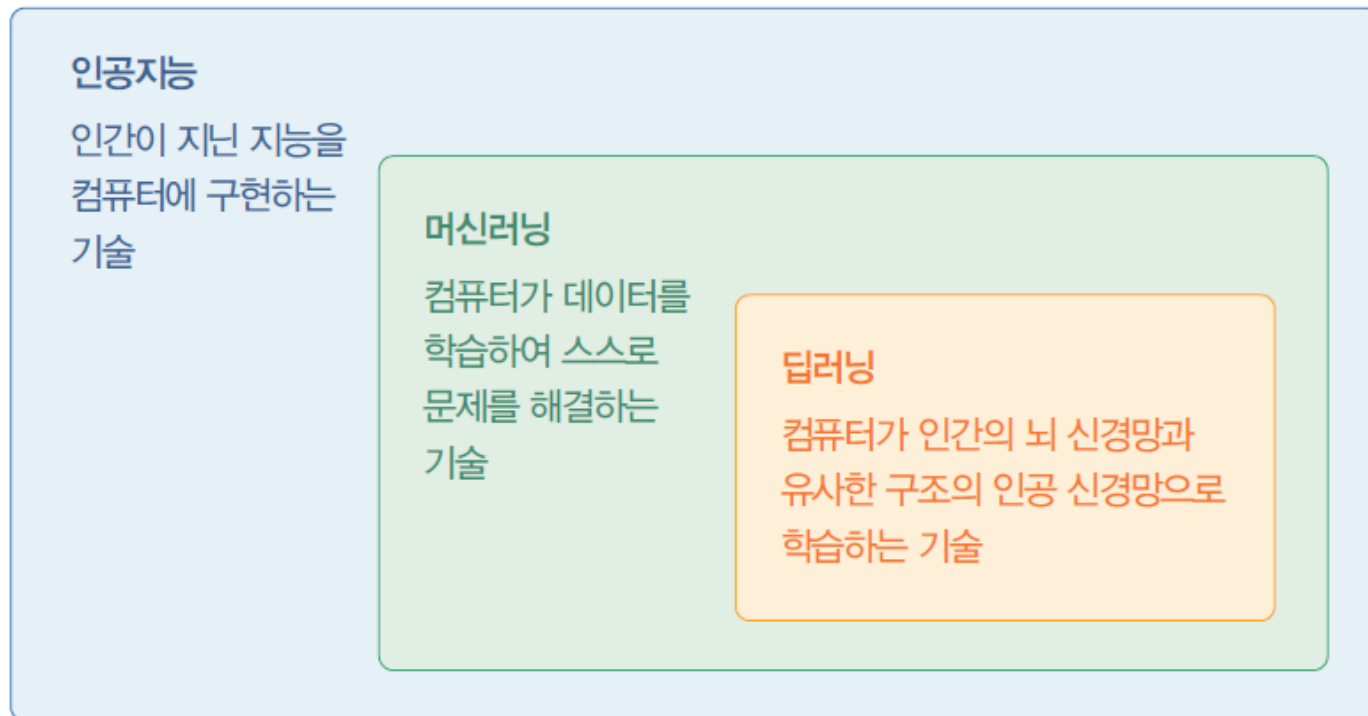


그림 13-1 인공지능, 머신러닝, 딥러닝의 관계

1. 머신러닝의 개요

■ 머신러닝의 개념

■ 머신러닝의 동작 과정

- 훈련 데이터로 머신러닝 모델을 학습시키면 데이터의 규칙이 생성되어 머신러닝 모델이 완성됨
- 모델에 새로운 데이터를 입력하면 결과를 예측할 수 있음
- 이미지를 분류하는 머신러닝 모델을 고양이와 강아지 이미지 데이터로 학습시키고, 완성된 모델에 새로운 고양이 이미지를 입력하면 고양이인지 강아지인지 분류

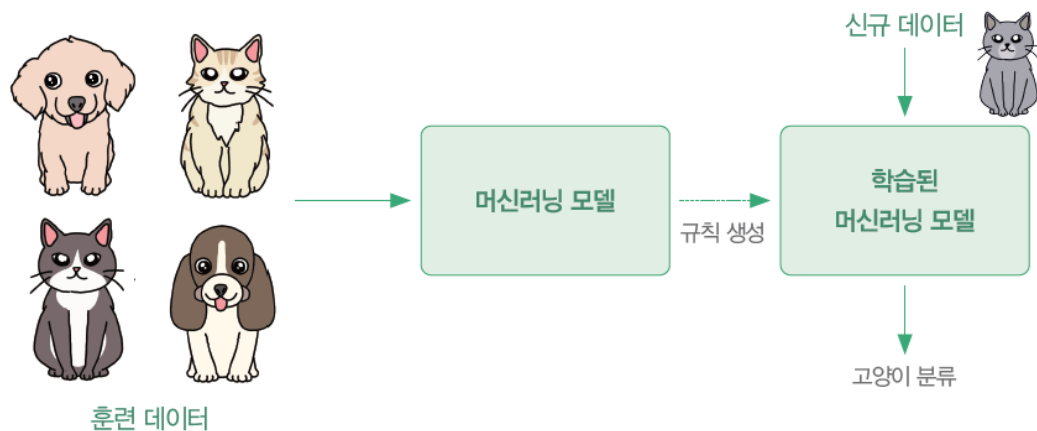


그림 13-2 머신러닝의 동작 과정

1. 머신러닝의 개요

■ 머신러닝의 종류

- 머신러닝은 훈련 데이터와 함께 주어지는 정보에 따라 지도학습과 비지도학습으로 분류

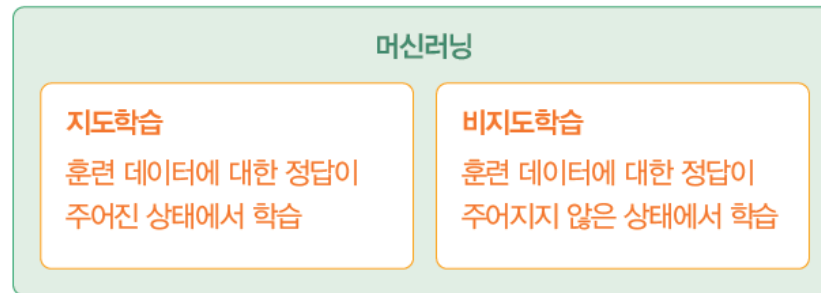


그림 13-3 머신러닝의 종류

1. 머신러닝의 개요

■ 머신러닝의 종류

■ 지도학습

- 지도학습(supervised learning): 데이터 분류(예측)의 결과가 주어진 상태에서 학습하는 방법
- 예를 들어 강아지와 고양이 이미지를 분류하는 머신러닝 프로그램을 개발할 때 모델에 이미지와 이미지에 대한 분류 결과(강아지 또는 고양이)를 함께 제공해서 학습시킴

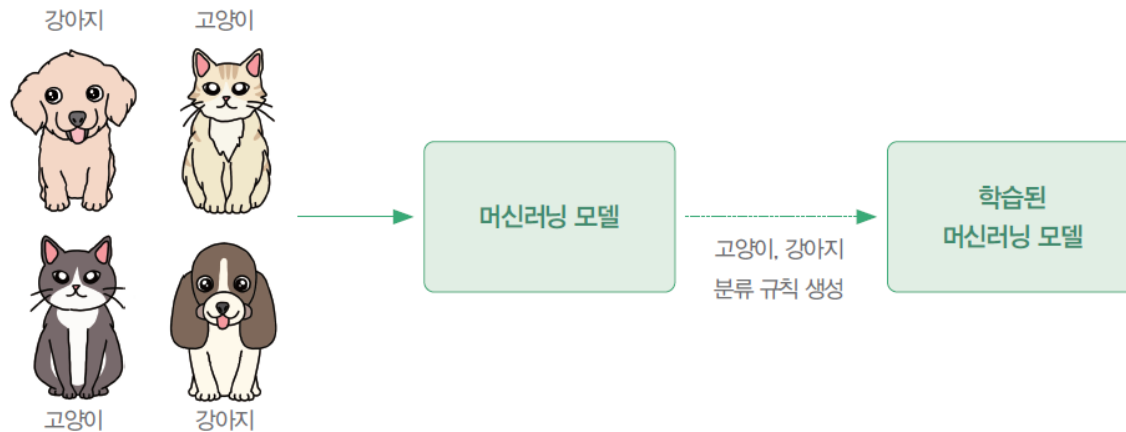


그림 13-4 지도학습

1. 머신러닝의 개요

■ 머신러닝의 종류

■ 지도학습

- 지도학습은 다시 분류와 회귀로 나눌 수 있음
- 분류(classification): 남자와 여자로 구분되는 성별과 같은 범주형 값을 예측하는 방법
- 회귀(regression): 가격이나 연비와 같은 연속적인 값을 예측하는 방법

1. 머신러닝의 개요

■ 머신러닝의 종류

■ 비지도학습

- 비지도학습unsupervised learning 은 데이터 분류(예측) 결과가 주어지지 않은 상태에서 학습하는 방법
- 예를 들어 강아지와 고양이 이미지를 분류하는 머신러닝 프로그램을 개발할 때 모델에 정답을 제공하지 않고 이미지만 학습시킴
- 모델은 이미지의 유사한 특성을 스스로 발견하여 분류

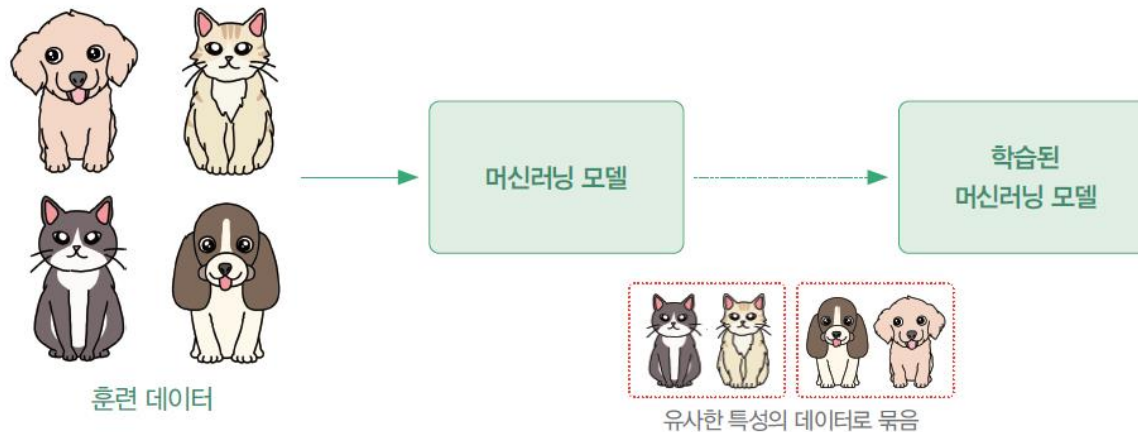


그림 13-5 비지도학습

1. 머신러닝의 개요

■ 머신러닝의 종류

■ 머신러닝의 절차



그림 13-6 머신러닝의 절차

- ① **데이터 수집 단계**는 문제를 해결하기 위한 데이터를 수집하는 단계. 데이터의 크기와 질을 고려해서 데이터를 수집해야 함
- ② **데이터 전처리 단계**는 수집한 데이터를 학습에 적합한 형태로 가공하는 단계. 데이터에 결측값이 있다면 처리하고, 데이터를 훈련 세트와 테스트 세트로 나눔. 훈련 세트(training set) 는 모델 학습에 활용할 데이터, 테스트 세트(test set) 는 모델의 성능을 평가할 때 사용하는 데이터

1. 머신러닝의 개요

■ 머신러닝의 종류

■ 머신러닝의 절차

- ③ **모델 생성 단계**는 문제를 해결하는 머신러닝 모델을 생성하는 단계. 문제에 적합한 모델을 선택하고 훈련 세트를 이용해서 선택한 모델을 학습시키면 모델이 완성됨
- ④ **모델 평가 단계**는 모델의 성능을 평가하는 단계. 주로 정확도를 기준으로 성능을 평가

머신러닝 분류

2. 머신러닝 분류

■ k-최근접 이웃 알고리즘

- k-최근접 이웃(k-Nearest Neighbors, k-NN) 알고리즘은 거리가 가까운 데이터 끼리는 속성이 유사할 것이라는 가정을 기반으로 하며 주로 머신러닝의 분류 알고리즘으로 사용됨
- 새로운 데이터를 분류할 때 기존 데이터 중 새로운 데이터와 거리가 가까운 이웃 데이터 k개를 이용하는 방식

2. 머신러닝 분류

■ k-최근접 이웃 알고리즘

- [그림 13-7]에서 새로운 데이터를 초록 원으로 표시
- 데이터를 파란 사각형 또는 빨간 삼각형으로 분류하기
- k-최근접 이웃 알고리즘을 이용한다면 초록색 원과 가까운 데이터가 어떤 색인지 확인하여 문제를 해결

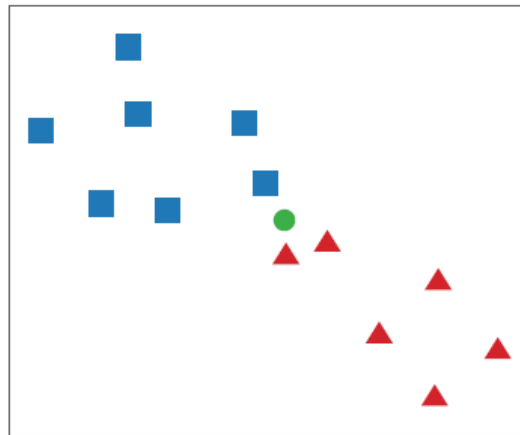
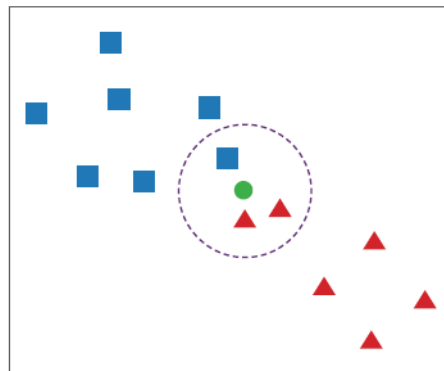


그림 13-7 기존 데이터와 새로운 데이터

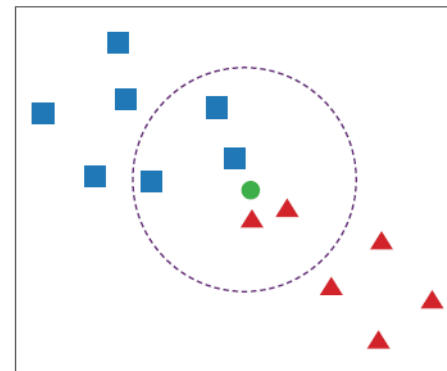
2. 머신러닝 분류

■ k-최근접 이웃 알고리즘

- 만약 k 가 3이면 초록색 원과 가장 가까운 데이터 3개를 확인
- 3개 중 빨간 삼각형이 가장 많으므로 새 데이터는 빨간 삼각형으로 분류
- 반면 k 가 5이면 초록색 원과 가장 가까운 데이터 5개를 확인
- 5개 중 파란 사각형이 가장 많으므로 새 데이터는 파란 사각형으로 분류



(a) k 가 3일 때



(b) k 가 5일 때

그림 13-8 k-최근접 이웃 알고리즘

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

- k-최근접 이웃 알고리즘을 이용해서 붓꽃iris의 품종을 분류하는 머신러닝 모델을 만들어보기
- 붓꽃을 세토사(setosa), 베르시칼라(versicolor), 버지니카(virginica) 품종으로 분류



(a) 세토사



(b) 베르시칼라



(c) 버지니카

그림 13-9 붓꽃의 품종

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

- 영국의 통계학자 로널드 피셔(Ronald Fisher) 교수가 1936년에 논문을 통해 붓꽃 데이터 세트를 소개했는데 머신러닝 분류에 대표적으로 사용되고 있음
- 이 데이터 세트는 실제로 측정한 데이터이고, 꽃받침 길이(sepal length), 꽃받침 너비(sepal width), 꽃잎 길이(petal length), 꽃잎 너비(petal width), 품종이 포함되어 있음

표 13-1 붓꽃 데이터 세트에 포함된 데이터

(단위: cm)

꽃받침 길이	꽃받침 너비	꽃잎 길이	꽃잎 너비	품종
5.1	3.5	1.4	0.2	세토사
4.9	3	1.4	0.2	세토사

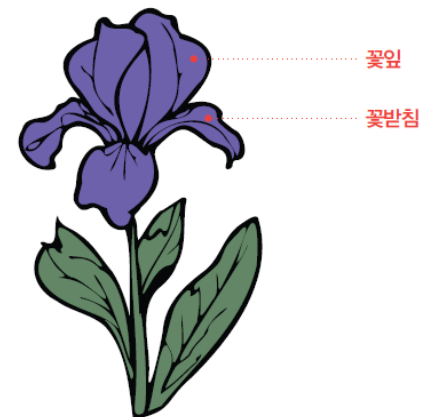


그림 13-10 붓꽃의 꽃잎과 꽃받침

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

- 꽃받침 길이, 꽃받침 너비, 꽃잎 길이, 꽃잎 너비 값으로 붓꽃의 품종을 예측하는 머신러닝 모델을 만들고자 함
- 이 모델은 하나의 붓꽃 데이터(꽃받침 길이, 꽃받침 너비, 꽃잎 길이, 꽃잎 너비)를 입력받아 품종을 예측
- 모델에 입력될 하나의 붓꽃 데이터, 즉 행을 샘플(sample)이라 하고 모델로부터 나오는 값인 품종을 출력이라고 함
- 그리고 샘플의 각 속성, 즉 열을 특성(feature)이라고 함

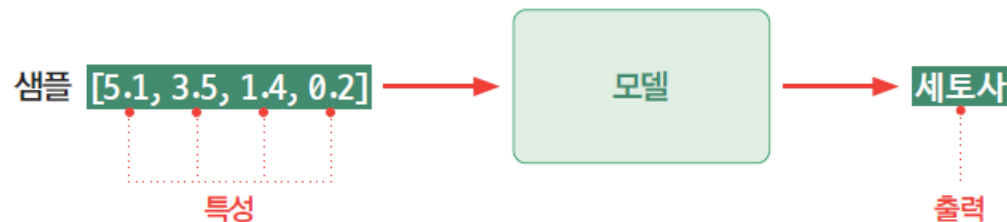


그림 13-11 붓꽃 분류 머신러닝 모델의 구조

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

- 분류 문제에서 붓꽃의 품종과 같이 모델이 출력할 수 있는 값의 집합을 클래스(class)라 하고, 특정 샘플의 클래스를 레이블(label)이라고 함
- 레이블은 모델이 예측해야 하는 값을 의미하는데 타깃(target)이라고도 하며 이 책에서는 레이블과 타깃을 모두 사용

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 수집하기

- 사이킷런 라이브러리에서 제공하는 붓꽃 데이터 세트를 활용
- 붓꽃 데이터 세트를 불러와서 iris에 저장

코드 13-1

```
01 from sklearn import datasets
02 iris = datasets.load_iris()
```

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 수집하기

- iris는 딕셔너리와 유사하게 키와 값으로 구성됨

코드 13-2

```
01 print('iris 키:', iris.keys())
```

```
iris 키: dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names',  
'filename', 'data_module'])
```

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 수집하기

- iris['data']는 붓꽃 150개의 꽃받침 길이, 꽃받침 너비, 꽃잎 길이, 꽃잎 너비 값
- 이는 품종을 예측하기 위해 활용하는 데이터인 특성feature에 해당하는데, 붓꽃 5개의 데이터만 확인하는 코드는 다음과 같음

코드 13-3

```
01 print('데이터:\n', iris['data'][:5])
```

데이터:

[[5.1 3.5 1.4 0.2]

[4.9 3. 1.4 0.2]

[4.7 3.2 1.3 0.2]

[4.6 3.1 1.5 0.2]

[5. 3.6 1.4 0.2]]

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 수집하기

- 실행 결과에서 각 행은 개별 붓꽃의 데이터이자 샘플에 해당

5.1	3.5	1.4	0.2	샘플
4.9	3	1.4	0.2	
4.7	3.2	1.3	0.2	

그림 13-12 iris['data']의 샘플

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 수집하기

- 샘플마다 특성이 4개인데 특성이 무엇을 의미하는지는 [코드 13-4]와 같이 `iris['feature_names']`로 특성 이름을 출력해서 확인할 수 있음

코드 13-4

```
01 print('특성 이름:', iris['feature_names'])
```

```
특성 이름: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

5.1	3.5	1.4	0.2
4.9	3	1.4	0.2
4.7	3.2	1.3	0.2

그림 13-13 특성 이름

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 수집하기

- iris['target']은 샘플 붓꽃의 품종을 의미하는 타깃
- 다음은 샘플 5개의 타깃을 확인하는 코드

코드 13-5

```
01 print('타깃:\n', iris['target'][:5])
```

```
[0 0 0 0 0]
```

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 수집하기

- 타깃 값 0, 1, 2가 각각 어느 품종을 나타내는지는 iris['target_names']로 타깃 이름을 출력하면 알 수 있음

코드 13-6

```
01 print('타깃 이름:', iris['target_names'])
```

```
타깃 이름: ['setosa' 'versicolor' 'virginica']
```

표 13-2 타깃과 붓꽃 품종

레이블	0	1	2
붓꽃 품종	setosa	versicolor	virginica

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 수집하기

- iris['data']와 iris['target']의 크기를 확인해 보면 iris['data']는 150행 4열이고 iris['target']은 150개

코드 13-7

```
01 print('데이터 크기:', iris['data'].shape)
02 print('타겟 크기:', iris['target'].shape)
```

```
데이터 크기: (150, 4)
타겟 크기: (150,)
```

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 수집하기

- iris['data']와 iris['target']의 관계를 살펴보기
- iris['data'] 특성은 붓꽃의 꽃받침 길이, 꽃받침 너비, 꽃잎 길이, 꽃잎 너비 값이고 iris['target']은 붓꽃의 품종
- [그림 13-14]에서 iris['data']의 첫 번째 행 5.1, 3.5, 1.4, 0.2 값을 갖는 붓꽃은 품종이 iris ['target']의 첫 번째 데이터인 0이라는 의미

iris['data']				iris['target']		
5.1	3.5	1.4	0.2	0	0	0
4.9	3	1.4	0.2			
4.7	3.2	1.3	0.2			

그림 13-14 특성과 타겟의 관계

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 전처리하기

- 데이터 세트를 훈련 세트와 테스트 세트로 나누어야 함
- 훈련 세트(training set)는 모델 학습에 필요한 데이터이고 테스트 세트(test set)는 모델이 얼마나 잘 작동하는지 측정하는 데 사용할 데이터
- 사이킷런의 train_test_split 함수를 이용하면 데이터 세트의 특성과 타깃을 훈련세트의 특성과 타깃, 테스트 세트의 특성과 타깃으로 나눌 수 있음

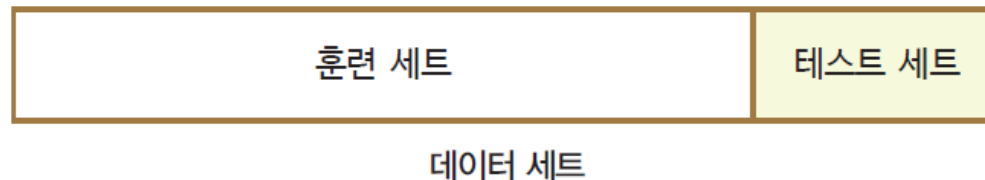


그림 13-15 데이터 세트 분할

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 전처리하기

- `train_test_split` 함수는 입력 데이터를 무작위로 섞은 후 전체 데이터의 75%를 훈련 세트로 정하고 나머지 25%를 테스트 세트로 분할
- 테스트 세트의 비율을 20%로 설정하고 싶으면 `test_size=0.2` 옵션을 추가하면 됨
- 만약 `random_state` 매개변수를 사용해서 난수 초깃값을 고정된 값으로 설정하면 여러 번 실행해도 분할 결과가 동일하게 나타남

| Syntax | `train_test_split()` 함수

```
from sklearn.model_selection import train_test_split  
train_test_split(특성, 타겟)
```

'특성'과 '타겟'을 각각 훈련 세트 75%와 테스트 세트 25%로 나눈다.

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 전처리하기

코드 13-8

```
01 from sklearn.model_selection import train_test_split
02
03 X_train, X_test, y_train, y_test = train_test_split(iris['data'], iris['target'],\
04                                             random_state=0)
05 print(X_train.shape, y_train.shape)
06 print(X_test.shape, y_test.shape)
```

(112, 4) (112,)

(38, 4) (38,)

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

■ 데이터 전처리하기

- X_train과 y_train은 훈련 세트이고 X_test와 y_test는 테스트 세트가 됨
- 그리고 X_train과 X_test는 특성이 되고 y_train과 y_test는 타겟이 됩니다. 특성은 붓꽃 품종을 예측하기 위해 활용하는 꽃받침 길이, 꽃받침 너비, 꽃잎 길이, 꽃잎 너비 데이터이고 타겟은 예측할 값인 품종

```
X_train, X_test, y_train, y_test = train_test_split(iris['data'], iris['target'], random_state=0)
```

테스트 세트의 특성 테스트 세트의 타겟

훈련 세트의 특성 훈련 세트의 타겟

데이터 세트의 특성 데이터 세트의 타겟

그림 13-16 iris 데이터 세트 분할 코드의 구조

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

▪ 모델 생성하기

- 붓꽃 품종을 예측하는 머신러닝 모델을 만들어보기
- 사이킷런의 KNeighborsClassifier 함수를 이용해서 k-최근접 이웃 모델을 설정할 수 있음
- n_neighbors 매개변수를 이용해서 이웃 개수를 지정할 수 있는데, 매개변수를 생략하면 이웃 개수가 5로 설정됨

| Syntax | KNeighborsClassifier() 함수

```
from sklearn.neighbors import KNeighborsClassifier  
KNeighborsClassifier(n_neighbors=이웃 개수)
```

k-최근접 이웃 모델을 설정한다.

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

▪ 모델 생성하기

- n_neighbors 매개변수를 3으로 한 k-최근접 이웃 모델을 설정하여 knn에 저장
- Knn은 k-최근접 이웃 알고리즘으로 동작하는 모델이 됨

코드 13-9

```
01 from sklearn.neighbors import KNeighborsClassifier
02
03 knn = KNeighborsClassifier(n_neighbors=3)
```

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

▪ 모델 생성하기

- 모델을 설정하면 훈련 세트의 특성과 타깃을 이용해서 모델을 학습시켜야 모델이 완성됨
- 모델 학습은 fit 함수를 이용하면 되는데, 특성을 2차원 배열 구조로 입력

| Syntax | fit() 함수

```
모델.fit(특성, 타깃)
```

‘모델’을 학습시킨다.

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

▪ 모델 생성하기

- 훈련 세트의 특성인 X_{train} 과 타깃인 y_{train} 을 이용해서 knn 모델을 학습시키면 모델이 완성됨
- 특성인 X_{train} 은 112행 4열인 2차원 배열 구조이므로 그대로 입력

코드 13-10

```
01 knn.fit(X_train, y_train)
```

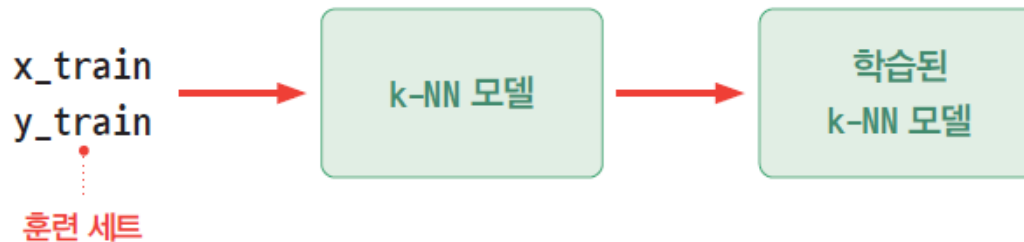


그림 13-17 모델 학습 과정

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

▪ 모델을 이용해 예측하기

- 생성한 모델을 이용해서 새로운 특성을 보고 붓꽃 품종을 예측할 수 있음
- 모델을 이용해서 예측할 때 predict 함수를 이용하는데, 이때 특성을 매개변수로 해야 함
- fit 함수와 마찬가지로 특성은 2차원 배열 구조로 입력

| Syntax | predict() 함수

```
모델.predict(특성)
```

‘모델’을 이용해서 예측한다.

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

▪ 모델을 이용해 예측하기

- 꽃받침 길이가 6cm, 꽃받침 너비가 3.5cm, 꽃잎 길이가 5cm, 꽃잎 너비가 1.5cm인
- 붓꽃을 어느 품종으로 예측하는지 알아보기
- 샘플이 하나이므로 1행 4열의 2차원 배열 `X_new`를 생성
- `predict` 함수의 매개변수인 특성은 2차원 배열이어야 하기 때문

코드 13-11

```
01 import numpy as np
02
03 X_new = np.array([[6, 3.5, 5, 1.5]])
```

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

▪ 모델을 이용해 예측하기

- 특성 X_{new} 를 보고 predict 함수를 이용해서 예측한 결과는 1

코드 13-12

```
01 pred = knn.predict(X_new)
02 print(pred)
```

[1]

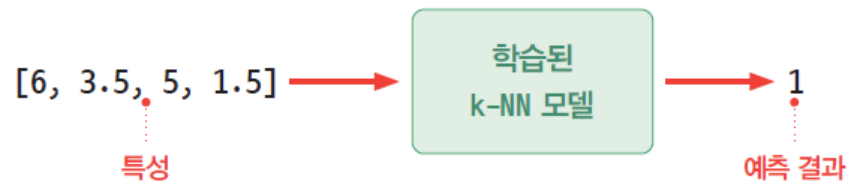


그림 13-18 학습된 모델의 예측 결과

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

▪ 모델을 이용해 예측하기

- 품종 1의 이름을 확인하는 코드는 [코드 13-13]과 같음

코드 13-13

```
01 print(iris['target_names'][pred])
```

```
['versicolor']
```

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

▪ 모델을 이용해 예측하기

- iris['target_names']는 타깃 값 0, 1, 2에 해당하는 품종 이름
- 예측 결과인 pred가 1일때 iris['target_names'][pred]는 iris['target_names'][1]이 되어 'versicolor'가 출력

'setosa'	'versicolor'	'virginica'
----------	--------------	-------------

`iris['target_names'][0]` `iris['target_names'][1]` `iris['target_names'][2]`

그림 13-19 품종 이름을 구하는 과정

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

▪ 모델을 이용해 예측하기

- 테스트 세트의 특성인 X_{test} 로 붓꽃 품종을 예측하는 코드는 [코드 13-14]와 같음

코드 13-14

```
01 y_pred = knn.predict(X_test)
02 print('테스트 세트 예측 값:\n', y_pred)
03 print('테스트 세트 실제 타겟:\n', y_test)
```

테스트 세트 예측 값:

```
[2 1 0 2 0 2 0 1 1 1 2 1 1 1 1 0 1 1 0 0 2 1 0 0 2 0 0 1 1 0 2 1 0 2 2 1 0 2]
```

테스트 세트 실제 타겟:

```
[2 1 0 2 0 2 0 1 1 1 2 1 1 1 1 0 1 1 0 0 2 1 0 0 2 0 0 1 1 0 2 1 0 2 2 1 0 1]
```

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

▪ 모델 평가하기

- 머신러닝 모델의 성능을 평가할 때 주로 정확도를 이용
- 여기서 정확도는 실제 타겟과 모델을 이용한 예측 값이 같은 경우의 비율

| Syntax | `score()` 함수

```
모델.score(특성, 타겟)
```

'모델'의 정확도를 계산한다.

2. 머신러닝 분류

■ 붓꽃 품종 분류하기

▪ 모델 평가하기

- 붓꽃 분류 모델에서 훈련 세트와 테스트 세트의 정확도를 구하는 코드는 [코드 13-15]와 같음

코드 13-15

```
01 print(f'훈련 세트 정확도: {knn.score(X_train, y_train):.3f}')
```

```
02 print(f'테스트 세트 정확도: {knn.score(X_test, y_test):.3f}')
```

```
훈련 세트 정확도: 0.964  
테스트 세트 정확도: 0.974
```

2. 머신러닝 분류

LAB 와인 분류하기

- 와인을 분류하는 머신러닝 프로그램을 만들어보기
- 사이킷런 라이브러리에서 제공하는 와인 데이터 세트를 활용
- 이 데이터 세트는 이탈리아의 한 지역에서 세 명의 경작자가 재배한 포도로 만든 와인을 화학적으로 분석한 결과

표 13-3 와인 데이터셋의 특성

특성 이름	설명
alcohol	알코올 함량
malic_acid	말산 함량
ash	회분 함량
alcalinity_of_ash	회분의 알칼리도
magnesium	마그네슘 함량
total_phenols	총 폴리페놀 함량

2. 머신러닝 분류

LAB 와인 분류하기

flavanoids	플라보노이드 함량
nonflavanoid_phenols	비플라보노이드 폴리페놀 함량
proanthocyanins	프로안토시아닌 함량
color_intensity	색상 강도
hue	색조
OD280/OD315_of_diluted_wines	희석된 와인의 OD280/OD315 비율
proline	프롤린 함량

2. 머신러닝 분류

LAB 와인 분류하기

- 다음은 머신러닝 분류 모델을 이용해서 와인을 분류하는 프로그램

코드 13-16

```
01 from sklearn import datasets
02 from sklearn.model_selection import train_test_split
03 from sklearn.neighbors import KNeighborsClassifier
04
05 # 데이터 수집하기
06 wine = datasets.load_wine()
07 print('wine 키:', wine.keys())
08 print('데이터:\n', wine['data'][:5])
09 print('특성 이름:', wine['feature_names'])
10 print('타겟:\n', wine['target'][:5])
11 print('타겟 이름:', wine['target_names'])
12 print('데이터 크기:', wine['data'].shape)
13 print('타겟 크기:', wine['target'].shape)
14
```


2. 머신러닝 분류

LAB 와인 분류하기

```
15 # 데이터 전처리하기
16 X_train, X_test, y_train, y_test = train_test_split(wine['data'], wine['target'],\
17                                                     random_state=0)
18 print(X_train.shape, y_train.shape)
19 print(X_test.shape, y_test.shape)
20
21 # 모델 생성하기
22 knn = KNeighborsClassifier(n_neighbors=3)
23 knn.fit(X_train, y_train)
24
25 # 모델 이용해 예측하기
26 y_pred = knn.predict(X_test)
27 print('테스트 세트 예측 값:\n', y_pred)
28 print('테스트 세트 실제 타겟:\n', y_test)
29
30 # 모델 평가하기
31 print(f'훈련 세트 정확도: {knn.score(X_train, y_train):.3f}')
32 print(f'테스트 세트 정확도: {knn.score(X_test, y_test):.3f}')
```

2. 머신러닝 분류

LAB 와인 분류하기

```
wine 키: dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names'])
```

데이터:

```
[[1.423e+01 1.710e+00 2.430e+00 1.560e+01 1.270e+02 2.800e+00 3.060e+00  
 2.800e-01 2.290e+00 5.640e+00 1.040e+00 3.920e+00 1.065e+03]  
[1.320e+01 1.780e+00 2.140e+00 1.120e+01 1.000e+02 2.650e+00 2.760e+00  
 2.600e-01 1.280e+00 4.380e+00 1.050e+00 3.400e+00 1.050e+03]  
[1.316e+01 2.360e+00 2.670e+00 1.860e+01 1.010e+02 2.800e+00 3.240e+00  
 3.000e-01 2.810e+00 5.680e+00 1.030e+00 3.170e+00 1.185e+03]  
[1.437e+01 1.950e+00 2.500e+00 1.680e+01 1.130e+02 3.850e+00 3.490e+00  
 2.400e-01 2.180e+00 7.800e+00 8.600e-01 3.450e+00 1.480e+03]]  
[1.324e+01 2.590e+00 2.870e+00 2.100e+01 1.180e+02 2.800e+00 2.690e+00  
 3.900e-01 1.820e+00 4.320e+00 1.040e+00 2.930e+00 7.350e+02]]
```

```
특성 이름: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols',  
 'flavanoids', 'nonflavanoid_phenols', 'proanthocyanins', 'color_intensity', 'hue', 'od280/  
od315_of_diluted_wines', 'proline']
```

2. 머신러닝 분류

LAB 와인 분류하기

타겟:

[0 0 0 0 0]

타겟 이름: ['class_0' 'class_1' 'class_2']

데이터 크기: (178, 13)

타겟 크기: (178,)

(133, 13) (133,)

(45, 13) (45,)

테스트 세트 예측 값:

[0 1 1 0 1 1 0 1 1 1 0 1 0 2 0 1 0 0 1 0 1 0 1 1 1 0 1 1 1 2 2 0 0 1 0 0 0 0 1 1 0 2 0 1 1 1]

테스트 세트 실제 타겟:

[0 2 1 0 1 1 0 2 1 1 2 2 0 1 2 1 0 0 1 0 1 0 0 1 1 1 1 1 1 2 0 0 1 0 0 0 2 1 1 2 0 0 1 1 1]

훈련 세트 정확도: 0.887

테스트 세트 정확도: 0.733

머신러닝 회귀

3. 머신러닝 회귀

■ 회귀분석의 개념

- 회귀분석(regression analysis): 하나 이상의 독립 변수가 변할 때 종속 변수가 얼마나 변할 것인지, 즉 하나 이상의 독립 변수가 종속 변수에 미치는 영향력을 예측하는 분석 기법
- 회귀분석의 결과로 독립 변수와 종속 변수 사이의 구체적인 관계를 식으로 표현할 수 있고, 이 식을 회귀식이라 함

3. 머신러닝 회귀

■ 회귀분석의 개념

- 일간 평균 기온과 일간 아이스크림 판매량 데이터가 있고, 평균 기온이 26도일 때 아이스크림 판매량은 어떻게 될지 회귀분석을 이용해 예측해 보기

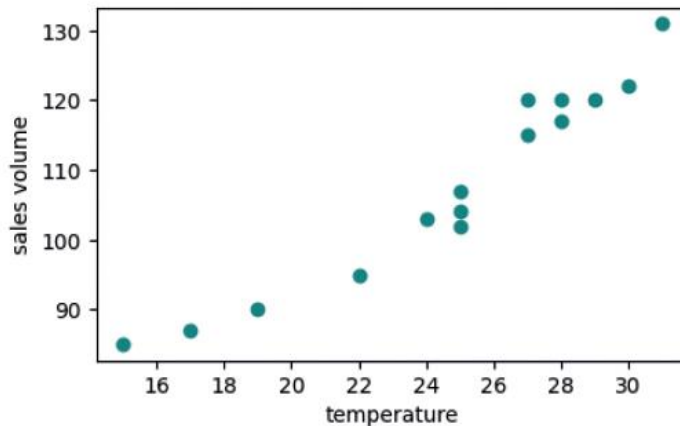
표 13-4 평균 기온과 아이스크림 판매량

평균 기온	아이스크림 판매량	평균 기온	아이스크림 판매량	평균 기온	아이스크림 판매량
15	85	28	117	28	120
17	87	25	104	27	115
22	95	27	120	31	131
25	102	24	103	29	120
19	90	30	122	25	107

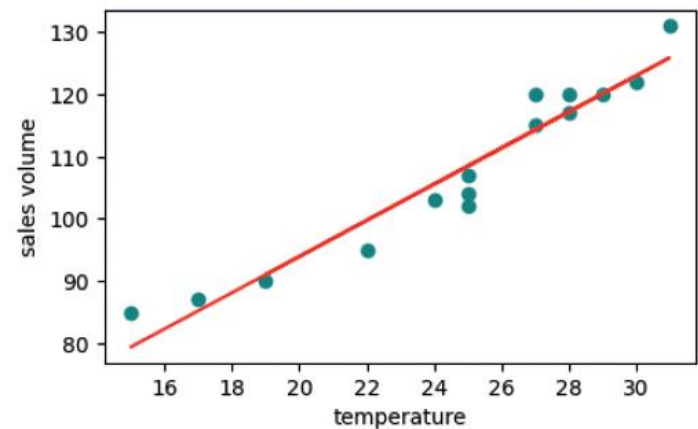
3. 머신러닝 회귀

■ 회귀분석의 개념

- 이 데이터를 [그림 13-20(a)]와 같이 x축이 평균 기온이고 y축이 아이스크림 판매량인 산점도로 나타냄
- 그리고 이 점들을 대표하는 직선을 [그림 13-20(b)]와 같이 그릴 수 있음



(a) 산점도



(b) 점들을 대표하는 직선

그림 13-20 평균 기온과 아이스크림 판매량의 산점도와 대표 직선

3. 머신러닝 회귀

■ 회귀분석의 개념

- 회귀분석은 이렇게 변수 간의 관계를 선으로 나타낼 수 있는 함수식을 찾는 분석
- 이 함수식을 회귀식이라 하고, 특히 직선의 회귀식을 찾는 회귀분석을 선형 회귀(linear regression) 라 함
- 예시를 이용해서 구한 회귀식은 다음과 같음

$$\text{아이스크림 판매량} = 35.9 + 2.9 \times (\text{평균 기온})$$

- 평균 기온은 독립 변수에 해당하고 아이스크림 판매량은 종속 변수에 해당
- 머신러닝에서는 독립 변수를 특성이라 하고 종속 변수를 타겟이라 함

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

- 캘리포니아 주택 가격을 예측하는 머신러닝 모델을 만드는 사례를 통해 머신러닝 회귀에 대해 더 구체적으로 살펴보기
- 이 절에서 사용할 캘리포니아 주택 가격 데이터 세트는 1990년 캘리포니아 인구 조사를 통해 얻은 평균 주택 가격 데이터 세트
- 캘리포니아 주택 가격 데이터 세트의 특성을 활용하여 주택 가격을 예측하는 머신러닝 모델을 만들어보기

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

■ 데이터 수집하기

- 사이킷런 라이브러리에서 제공하는 캘리포니아 주택 데이터 세트(California Housing dataset) 를 불러와서 housing에 저장
- 그리고 housing이 어떤 키로 이루어져 있는지 확인

코드 13-17

```
01 from sklearn import datasets
02 housing = datasets.fetch_california_housing()
03 print(housing.keys())
```

```
dict_keys(['data', 'target', 'frame', 'target_names', 'feature_names', 'DESCR'])
```

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

■ 데이터 수집하기

- housing['data']는 20,640개의 평균 소득, 평균 주택 연령, 가구당 평균 방 수 등의 값
- 이는 주택 가격을 예측하기 위해 활용하는 특성에 해당
- 주택 3개의 데이터만 확인하는 코드는 다음과 같음
- 실행 결과 각 행이 개별 주택의 특성을 의미

코드 13-18

```
01 print('데이터:\n', housing['data'][:3])
```

데이터:

```
[[ 8.32520000e+00  4.10000000e+01  6.98412698e+00  1.02380952e+00  
   3.22000000e+02  2.55555556e+00  3.78800000e+01 -1.22230000e+02]  
 [ 8.30140000e+00  2.10000000e+01  6.23813708e+00  9.71880492e-01  
   2.40100000e+03  2.10984183e+00  3.78600000e+01 -1.22220000e+02]  
 [ 7.25740000e+00  5.20000000e+01  8.28813559e+00  1.07344633e+00  
   4.96000000e+02  2.80225989e+00  3.78500000e+01 -1.22240000e+02]]
```

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

■ 데이터 수집하기

- 샘플의 특성이 8개인데 housing['feature_names']로 알아낸 특성의 이름과 그 의미는 [표 13-5]와 같음

코드 13-19

```
01 print('특성 이름:', housing['feature_names'])
```

```
특성 이름: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup',  
'Latitude', 'Longitude']
```

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

■ 데이터 수집하기

표 13-5 캘리포니아 주택 데이터 세트의 특성

특성 이름	설명
MedInc	블록 그룹의 평균 소득
HouseAge	블록 그룹의 평균 주택 연령
AveRooms	가구당 평균 방 수
AveBedrms	가구당 평균 침실 수
Population	블록 그룹의 인구
AveOccup	평균 가구 구성원 수
Latitude	블록 그룹의 위도
Longitude	블록 그룹의 경도

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

■ 데이터 수집하기

- housing['target']은 housing['data'] 주택의 가격
- 다음은 주택 5개의 타깃 값과 타깃 이름을 확인하는 코드
- 타깃 값은 MedHouseVal, 즉 평균 주택 가격

코드 13-20

```
01 print('타깃:\n', housing['target'][:5])  
02 print('타깃 이름:', housing['target_names'])
```

```
타깃:  
[4.526 3.585 3.521 3.413 3.422]  
타깃 이름: ['MedHouseVal']
```

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

■ 데이터 수집하기

- 특성과 타겟의 크기를 확인

코드 13-21

```
01 print('데이터 크기:', housing['data'].shape)
02 print('타겟 크기:', housing['target'].shape)
```

```
데이터 크기: (20640, 8)
타겟 크기: (20640,)
```

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

■ 데이터 수집하기

- 다음 그림에서 housing['data']의 첫 번째 행 데이터는 8.32520000e+00, 4.10000000e+01, ..., -1.22230000e+02이고, 주택 가격은 housing['target']의 첫 번째인 4.526

housing['data']				housing['target']	
8.325e+00	4.100e+01	...	-1.22e+02	4.526	3.585
8.301e+00	2.100e+01	...	-1.22e+02		
⋮					

그림 13-21 특성과 타겟의 관계

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

■ 데이터 전처리하기

- 사이킷런의 `train_test_split` 함수를 이용해서 특성인 `housing['data']`와 타깃인 `housing['target']`을 훈련 세트와 테스트 세트로 나눔

코드 13-22

```
01 from sklearn.model_selection import train_test_split
02
03 X_train, X_test, y_train, y_test = train_test_split(housing['data'], housing['target'],
04                                                    random_state=0)
05 print(X_train.shape, y_train.shape)
06 print(X_test.shape, y_test.shape)
```

```
(15480, 8) (15480,)
(5160, 8) (5160,)
```

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

■ 데이터 전처리하기

- X_train과 y_train은 훈련 세트이고 X_test와 y_test는 테스트 세트
- X_train과 X_test는 특성이고 y_train과 y_test는 타깃

```
X_train, X_test, y_train, y_test = train_test_split(housing['data'], housing['target'], ...)
```

테스트 세트의 특성 테스트 세트의 타깃

훈련 세트의 특성 훈련 세트의 타깃

데이터 세트의 특성 데이터 세트의 타깃

그림 13-22 housing 데이터 세트 분할 코드의 구조

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

▪ 모델 생성하기

- 주택 정보로 주택 가격을 예측하는 머신러닝 모델을 만들어보기
- 사이킷런에서 제공하는 LinearRegression 함수를 이용해서 선형 회귀 모델을 설정할 수 있음

| Syntax | LinearRegression() 함수

```
from sklearn.linear_model import LinearRegression  
LinearRegression()
```

선형 회귀 모델을 설정한다.

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

■ 모델 생성하기

- 선형 회귀 모델 역시 훈련 세트의 특성과 타깃을 이용해서 학습해야 모델이 완성
- fit 함수를 이용
- fit 함수의 특성인 X_train은 15,480행 8열의 2차원 배열 구조이므로 그대로 사용하면 됨

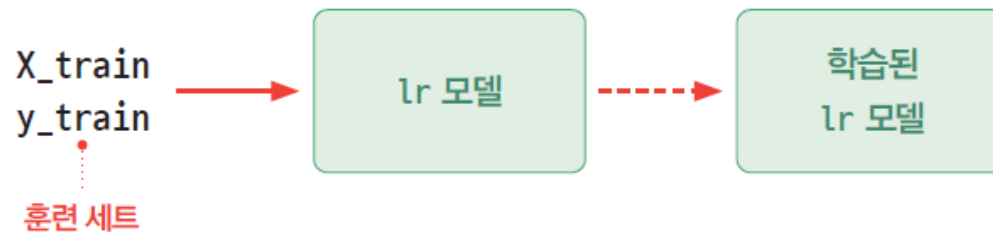


그림 13-23 선형 회귀 모델의 학습

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

▪ 모델 생성하기

코드 13-23

```
01 from sklearn.linear_model import LinearRegression
02
03 lr = LinearRegression()
04 lr.fit(X_train, y_train)
```

- 모델이 완성되면 특성($x_1, x_2, \dots$)의 변화량에 따라 타겟(y)이 어떻게 변하는지를 추정하여 식으로 나타낼 수 있고, 이 식을 회귀식이라 함

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

▪ 모델 생성하기

$$y = a + b_1x_1 + b_2x_2 + \dots$$

- 회귀식에서 a 를 절편intercept이라 하는데 값이 intercept_ 변수에 저장됨
- 회귀식에서 $b_1, b_2, \dots$ 는 계수coefficient 또는 가중치weight 라 하는데 값이 coef_에 리스트로 저장됨
- 다음은 절편과 계수를 확인하는 코드

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

▪ 모델 생성하기

코드 13-24

```
01 import numpy as np
02
03 print('절편:', lr.intercept_)
04 print('계수:', np.round(lr.coef_,3))
```

절편: -36.609593778714334

계수: [0.439 0.01 -0.103 0.617 -0. -0.004 -0.417 -0.431]

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

■ 모델 생성하기

- 회귀식의 특성, 타깃, 절편, 계수는 다음과 같음

$$\text{MedHouseVal} = -36.6 + 0.439 * \text{MedInc} + 0.01 * \text{HouseAge} - 0.103 * \text{AveRooms} + \dots$$

The diagram shows the equation $\text{MedHouseVal} = -36.6 + 0.439 * \text{MedInc} + 0.01 * \text{HouseAge} - 0.103 * \text{AveRooms} + \dots$ with red labels and dotted lines pointing to specific parts: '타깃' (target) points to 'MedHouseVal'; '절편' (intercept) points to '-36.6'; '특성1' (feature 1) points to 'MedInc'; '계수1' (coefficient 1) points to '0.439'; '특성2' (feature 2) points to 'HouseAge'; '계수2' (coefficient 2) points to '0.01'; '특성3' (feature 3) points to 'AveRooms'; '계수3' (coefficient 3) points to '-0.103'.

그림 13-24 선형 회귀식

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

■ 모델을 이용해서 예측하기

- 생성한 모델을 이용해서 테스트 세트의 특성인 X_{test} 에 대한 주택 가격을 예측하여 $pred$ 에 저장하고, $pred$ 에서 10개를 출력



그림 13-25 학습된 선형 회귀 모델을 이용하는 예측

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

▪ 모델을 이용해서 예측하기

- 테스트 세트의 예측 값과 실제 타깃을 출력해서 비교

코드 13-25

```
01 pred = lr.predict(X_test)
02 print('테스트 세트 예측 값 :\n', np.round(pred[:10],3))
03 print('테스트 세트 실제 타깃 :\n', y_test[:10])
```

테스트 세트 예측 값:

[2.278 2.796 1.909 1.026 2.959 2.097 2.994 3.067 2.986 2.39]

테스트 세트 실제 타깃:

[1.369 2.413 2.007 0.725 4.6 1.2 2.47 3.369 3.397 2.656]

예측값 2.278 2.796 1.909 ...

실제 타깃 1.369 2.413 2.007 ...

그림 13-26 예측 값과 실제 타깃

3. 머신러닝 회귀

■ 캘리포니아 주택 가격 예측하기

▪ 모델 평가

- score 함수를 이용해서 모델의 정확도를 측정

코드 13-26

```
01 print(f'훈련 세트 정확도: {lr.score(X_train, y_train):.3f}')
```

```
02 print(f'테스트 세트 정확도: {lr.score(X_test, y_test):.3f}')
```

```
훈련 세트 정확도: 0.611
```

```
테스트 세트 정확도: 0.591
```

3. 머신러닝 회귀

LAB 당뇨병 진행 정도 예측하기

- 당뇨병 진행률을 예측하는 프로그램을 만들어보기
- 사이킷런 라이브러리에서 제공하는 당뇨병 데이터 세트를 활용하고자 함
- 미국 피마 인디언 442명을 대상으로 혈압, 혈당, 콜레스테롤 등 10가지 특성과 당뇨병 진행률을 기록한 데이터
- 10가지 특성의 이름과 내용은 [표 13-6]과 같고 타겟은 1년 뒤에 측정한 당뇨병 진행률

3. 머신러닝 회귀

LAB 당뇨병 진행 정도 예측하기

표 13-6 당뇨병 데이터 세트의 특성

특성 이름	설명
age	나이
sex	성별
bmi	체질량지수
bp	평균 혈압
s1	혈중 총콜레스테롤
s2	저밀도 지질단백질
s3	고밀도 지질단백질
s4	총 콜레스테롤 수치
s5	혈중 트리글리세라이드 수치
s6	혈당 수치

3. 머신러닝 회귀

LAB 당뇨병 진행 정도 예측하기

- 다음은 머신러닝 회귀 모델을 이용해서 당뇨병 진행률을 예측하는 프로그램의 전체 코드와 실행 결과

코드 13-27

```
01 from sklearn import datasets
02 from sklearn.model_selection import train_test_split
03 from sklearn.linear_model import LinearRegression
04 import numpy as np
05
06 # 데이터 수집하기
07 diabetes = datasets.load_diabetes()
08 print('diabetes 키:', diabetes.keys())
09 print('데이터:\n', diabetes['data'][:3])
10 print('특성 이름:', diabetes['feature_names'])
11 print('타겟:\n', diabetes['target'][:3])
12 print('데이터 크기:', diabetes['data'].shape)
13 print('타겟 크기:', diabetes['target'].shape)
```

3. 머신러닝 회귀

LAB 당뇨병 진행 정도 예측하기

```
14
15 # 데이터 전처리하기
16 X_train, X_test, y_train, y_test = train_test_split(diabetes['data'],\
17                                                     diabetes['target'], random_state=0)
18 print(X_train.shape, y_train.shape)
19 print(X_test.shape, y_test.shape)
20
21 # 모델 생성하기
22 lr = LinearRegression()
23 lr.fit(X_train, y_train)
24
25 print('절편:', np.round(lr.intercept_,3))
26 print('계수:', np.round(lr.coef_,3))
27
28 # 모델 이용해 예측하기
29 pred = lr.predict(X_test)
30 print('테스트 세트 예측 값:\n', np.round(pred[:10],3))
31 print('테스트 세트 실제 타겟:\n', y_test[:10])
32
33 # 모델 평가하기
34 print(f'훈련 세트 정확도: {lr.score(X_train, y_train):.3f}')
35 print(f'테스트 세트 정확도: {lr.score(X_test, y_test):.3f}')
```

3. 머신러닝 회귀

LAB 당뇨병 진행 정도 예측하기

```
diabetes 키: dict_keys(['data', 'target', 'frame', 'DESCR', 'feature_names', 'data_filename',  
'target_filename', 'data_module'])  
데이터:  
[[ 0.03807591  0.05068012  0.06169621  0.02187239 -0.0442235 -0.03482076  
-0.04340085 -0.00259226  0.01990749 -0.01764613]  
[-0.00188202 -0.04464164 -0.05147406 -0.02632753 -0.00844872 -0.01916334  
 0.07441156 -0.03949338 -0.06833155 -0.09220405]  
[ 0.08529891  0.05068012  0.04445121 -0.00567042 -0.04559945 -0.03419447  
-0.03235593 -0.00259226  0.00286131 -0.02593034]]  
특성 이름: ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']  
타겟:  
[151.  75. 141.]  
데이터 크기: (442, 10)  
타겟 크기: (442,)  
(331, 10) (331,)  
(111, 10) (111,)  
절편: 153.068  
계수: [ -43.262 -208.666  593.407  302.891 -560.191  261.408  -8.867  135.932  703.184  
28.35 ]  
테스트 세트 예측 값:  
[241.845 250.119 164.968 119.116 188.232 260.559 113.078 190.543 151.887 236.505]  
테스트 세트 실제 타겟:  
[321. 215. 127.  64. 175. 275. 179. 232. 142.  99.]  
훈련 세트 정확도: 0.555  
테스트 세트 정확도: 0.359
```


[플러스 예제] MNIST 분류하기

4. [플러스 예제] MNIST 분류하기

- MNIST(Modified National Institute of Standards and Technology database):
0부터 9까지의 숫자를 손으로 쓴 이미지 70,000개를 모은 데이터 세트
- 데이터 세트는 머신러닝 분야의 'Hello World'라 불릴 정도로 많이 사용됨
- MNIST 데이터 세트를 활용하여 이미지를 분류하는 머신러닝 모델을 만들어보기

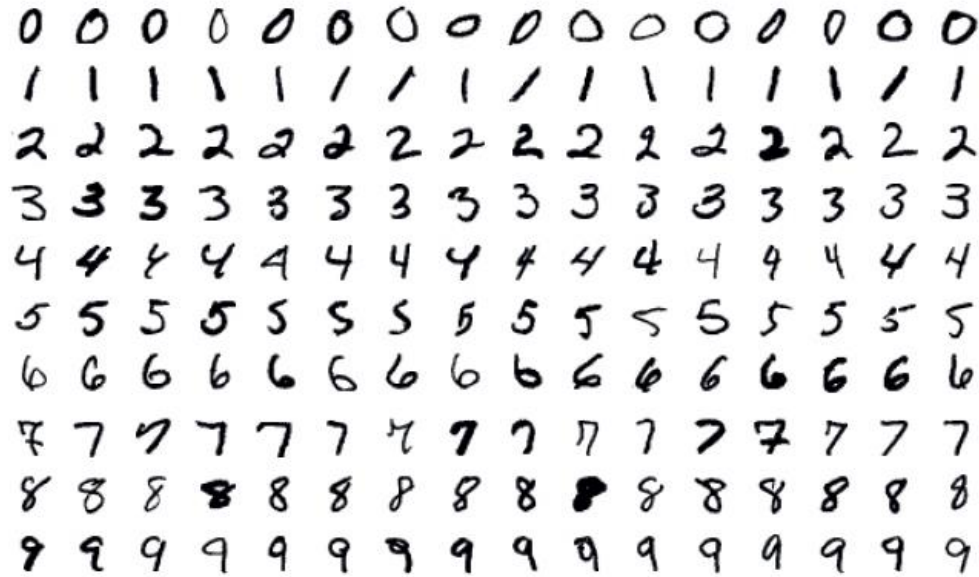


그림 13-27 MNIST 데이터 세트의 일부

4. [플러스 예제] MNIST 분류하기

■ 데이터 수집하기

- 사이킷런 라이브러리에서 제공하는 MNIST 데이터 세트를 불러와서 mnist에 저장
- 데이터가 넘파이 배열이므로 as_frame 매개변수를 False로 설정하고, 데이터를 원하는 형식으로 변환하는 파싱을 자동으로 선택하도록 parser 매개변수를 'auto'로 설정

```
01 from sklearn.datasets import fetch_openml
02 mnist = fetch_openml('mnist_784', as_frame=False, parser='auto')
```

4. [플러스 예제] MNIST 분류하기

■ 데이터 수집하기

- mnist 객체가 어떤 키로 이루어져 있는지 확인

```
01 print(mnist.keys())
```

```
dict_keys(['data', 'target', 'frame', 'categories', 'feature_names', 'target_names',  
'DESCR', 'details', 'url'])
```

4. [플러스 예제] MNIST 분류하기

■ 데이터 수집하기

- mnist['data']와 mnist['target']의 크기를 확인
- 특성인 mnist['data']는 70,000행 784열
- 여기서 784는 28×28 과 같고, 가로 28픽셀과 세로 28픽셀인 모든 이미지의 픽셀수
- 타깃인 mnist['target']은 이미지가 표현하는 숫자의 개수와 같은 70,000개

```
01 print('데이터 크기:', mnist['data'].shape)
02 print('타깃 크기:', mnist['target'].shape)
```

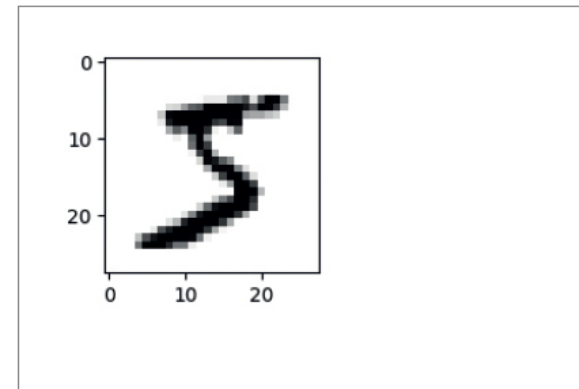
데이터 크기: (70000, 784)
타깃 크기: (70000,)

4. [플러스 예제] MNIST 분류하기

■ 데이터 수집하기

- mnist['data']의 첫 번째 이미지를 출력하기
- reshape 함수를 이용해서 이미지 크기를 784에서 28행 28열의 배열로 바꿔야 이미지로 출력됨
- cmap 매개변수를 이용해서 이미지 색상을 'binary'로 설정

```
01 import matplotlib.pyplot as plt
02
03 plt.figure(figsize=(2,2))
04 img = mnist['data'][0].reshape(28,28)
05 plt.imshow(img, cmap='binary')
06 plt.show()
```



4. [플러스 예제] MNIST 분류하기

■ 데이터 수집하기

- 숫자 5가 적힌 이미지가 나타나고, 이 이미지의 타깃 값을 출력하면5

```
01 print(mnist['target'][0])
```

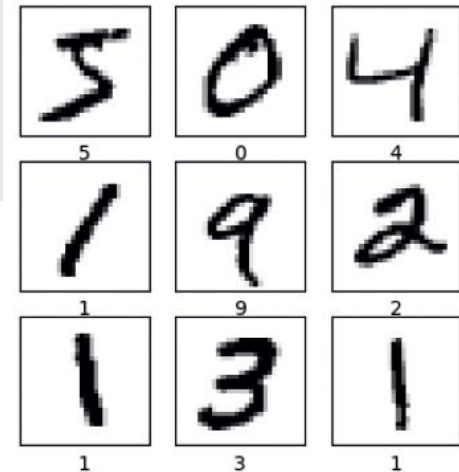
5

4. [플러스 예제] MNIST 분류하기

■ 데이터 수집하기

- 9개의 이미지를 3행 3열로 출력하고 이미지 아래에 타깃을 출력

```
01 plt.figure(figsize=(4,4))
02 for i in range(9):
03     plt.subplot(3,3,i+1)
04     plt.xticks([])
05     plt.yticks([])
06     plt.imshow(mnist['data'][i].reshape(28,28), cmap='binary')
07     plt.xlabel(mnist['target'][i])
08 plt.show()
```



4. [플러스 예제] MNIST 분류하기

■ 데이터 전처리하기

- 사이킷런의 `train_test_split` 함수를 이용해서 특성인 `mnist['data']`와 타깃인 `mnist['target']`을 훈련 세트인 `X_train`과 `y_train`, 테스트 세트인 `X_test`와 `y_test`로 나눔

```
01 from sklearn.model_selection import train_test_split
02
03 X_train, X_test, y_train, y_test = train_test_split(mnist['data'], mnist['target'],
    random_state=0)
04 print(X_train.shape, y_train.shape)
05 print(X_test.shape, y_test.shape)
```

```
(52500, 784) (52500,)
```

```
(17500, 784) (17500,)
```

4. [플러스 예제] MNIST 분류하기

■ 모델 생성하기

- k-최근접 이웃 모델을 설정하고 훈련 세트를 이용해서 모델을 학습시켜 숫자 이미지로 숫자를 분류하는 머신러닝 모델을 생성해 보기
- fit 함수의 특성인 X_train은 52,500행 784열의 2차원 배열 구조이므로 그대로 사용

```
01 from sklearn.neighbors import KNeighborsClassifier
02
03 knn = KNeighborsClassifier()
04 knn.fit(X_train, y_train)
```

4. [플러스 예제] MNIST 분류하기

■ 모델 이용해 예측하기

- 테스트 세트의 특성인 X_{test} 이미지가 나타내는 숫자를 예측하고 예측 값 10개를 확인
- 테스트 세트의 실제 타겟과 비교해 보면 10개 모두 동일한 것을 확인

```
01 y_pred = knn.predict(X_test)
02 print('테스트 세트 예측 값:\n', y_pred[:10])
03 print('테스트 세트 실제 타겟:\n', y_test[:10])
```

테스트 세트 예측 값:

['0' '4' '1' '2' '7' '9' '7' '1' '1' '7']

테스트 세트 실제 타겟:

['0' '4' '1' '2' '7' '9' '7' '1' '1' '7']

4. [플러스 예제] MNIST 분류하기

■ 모델 평가하기

- 모델의 훈련 세트와 테스트 세트 예측 정확도는 0.98과 0.969로 높은 정확도를 보임

```
01 print(f'훈련 세트 정확도: {knn.score(X_train, y_train):.3f}')
```

```
02 print(f'테스트 세트 정확도: {knn.score(X_test, y_test):.3f}')
```

훈련 세트 정확도: 0.980

테스트 세트 정확도: 0.969

4. [플러스 예제] MNIST 분류하기

■ 모델 평가하기

▪ 전체 프로그램

```
01 from sklearn.datasets import fetch_openml
02 from sklearn.model_selection import train_test_split
03 from sklearn.neighbors import KNeighborsClassifier
04
05 # 데이터 수집하기
06 mnist = fetch_openml('mnist_784', as_frame=False, parser='auto')
07
08 # 데이터 전처리하기
09 X_train, X_test, y_train, y_test = train_test_split(mnist['data'], mnist['target'],
10                                                    random_state=0)
```

4. [플러스 예제] MNIST 분류하기

■ 모델 평가하기

■ 전체 프로그램

```
11 print(X_train.shape, y_train.shape)
12 print(X_test.shape, y_test.shape)
13
14 # 모델 생성하기
15 knn = KNeighborsClassifier()
16 knn.fit(X_train, y_train)
17
18 # 모델 이용해 예측하기
19 y_pred = knn.predict(X_test)
20 print('테스트 세트 예측 값:\n', y_pred[:10])
21 print('테스트 세트 실제 타겟:\n', y_test[:10])
22
23 # 모델 평가하기
24 print(f'훈련 세트 정확도: {knn.score(X_train, y_train):.3f}')
25 print(f'테스트 세트 정확도: {knn.score(X_test, y_test):.3f}')
```

4. [플러스 예제] MNIST 분류하기

■ 모델 평가하기

■ 전체 프로그램

(52500, 784) (52500,)

(17500, 784) (17500,)

테스트 세트 예측 값:

['0' '4' '1' '2' '7' '9' '7' '1' '1' '7']

테스트 세트 실제 타겟:

['0' '4' '1' '2' '7' '9' '7' '1' '1' '7']

훈련 세트 정확도: 0.980

테스트 세트 정확도: 0.969